

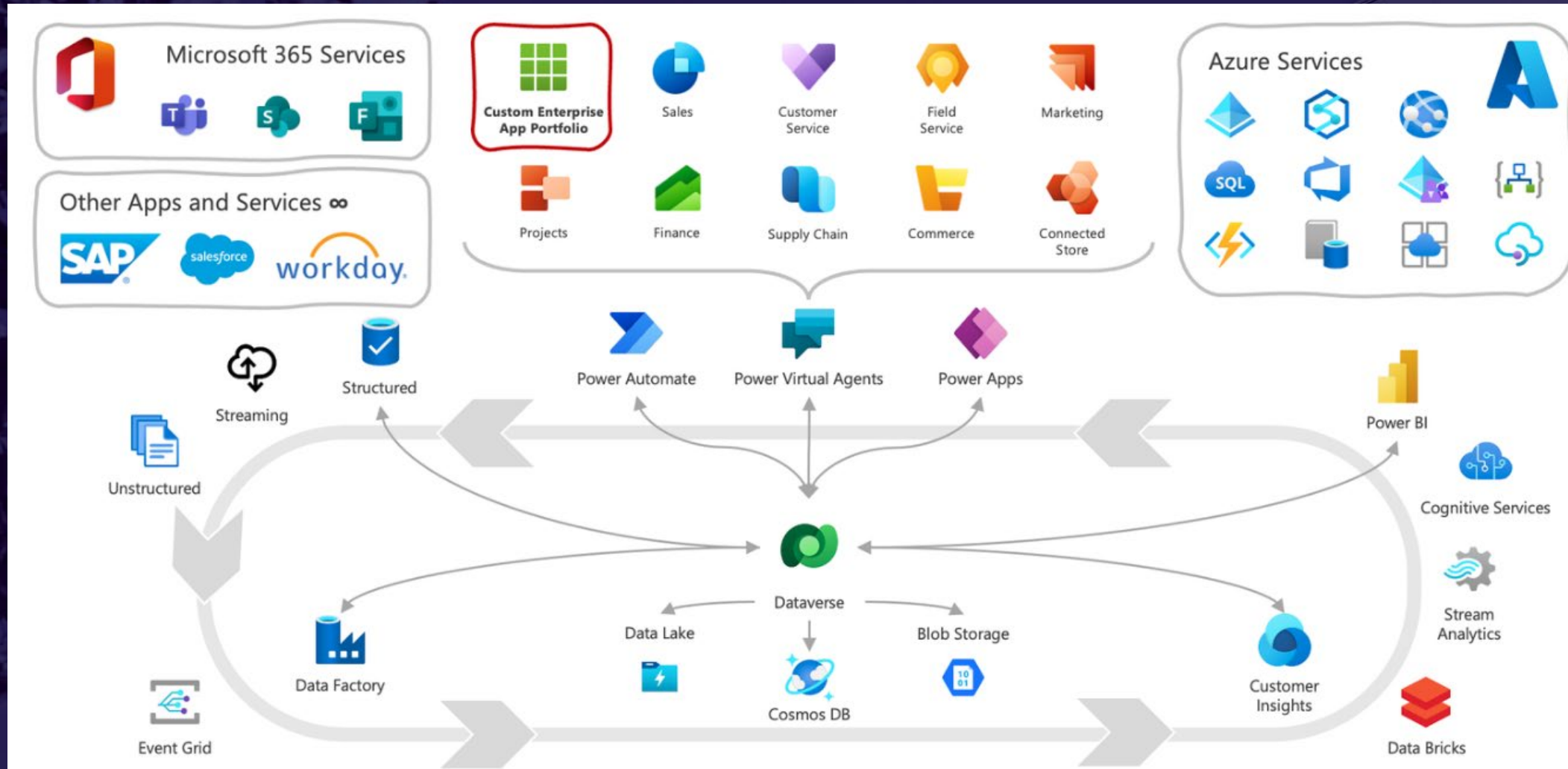


COM682

Cloud Native Development

- 1. Relational Databases*
- 2. Non-Relational Databases*
- 3. Azure SQL Deployments*

Modern Cloud Applications





Broadly, there are three classes of Data structure:

- > Structured Data
- > Semi-Structured Data
- > Unstructured Data

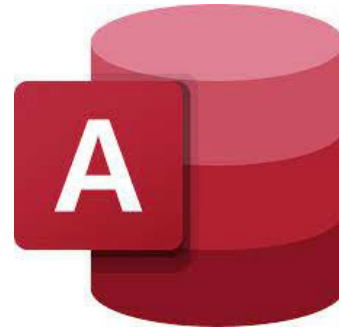


Identifying the class of data you are to handle, and selecting the appropriate data store can greatly help or hinder development efforts.

Relational Databases

Introduction

- Relational database management systems (RDBMS) were proposed in the 1970s by E.F. Codd
- This paradigm dominated database design until the turn of the century
- Examples of RDBMS include:
 - *MS SQL*
 - MySQL
 - MS Access
 - Oracle SQL



Relational Databases

Introduction

- Relational database management systems (RDBMS) have a number of defining characteristics
 - Data in a RDBMS is organised through the “**Relational Model**”
 - Support Structured Query Language (SQL) for interacting with a database
 - **Support relationships** between tables
 - Typically support **ACID**(Atomicity, Consistency, Isolation, and Durability) transactions. All changes to data are performed as if they are a single operation.

Relational Databases

Relational Model

Table also called **Relation**

Primary Key	Domain Ex: NOT NULL	
CustomerID	CustomerName	Status
1	Google	Active
2	Amazon	Active
3	Apple	Inactive

Tuple OR Row

Total # of rows is **Cardinality**

Column OR Attributes

Total # of column is **Degree**

Relational Databases

Relational Model

- Relational models are essentially tables within a database
- Tables generally represent one entity type, such as student/customer records
- These tables consist of rows and columns
 - Rows may be referred to as Tuples or Records
 - Columns may be referred to as Fields or Attributes
 - Tables may be referred to as Relations or Base Revlar*

Relational Databases

Relational Model

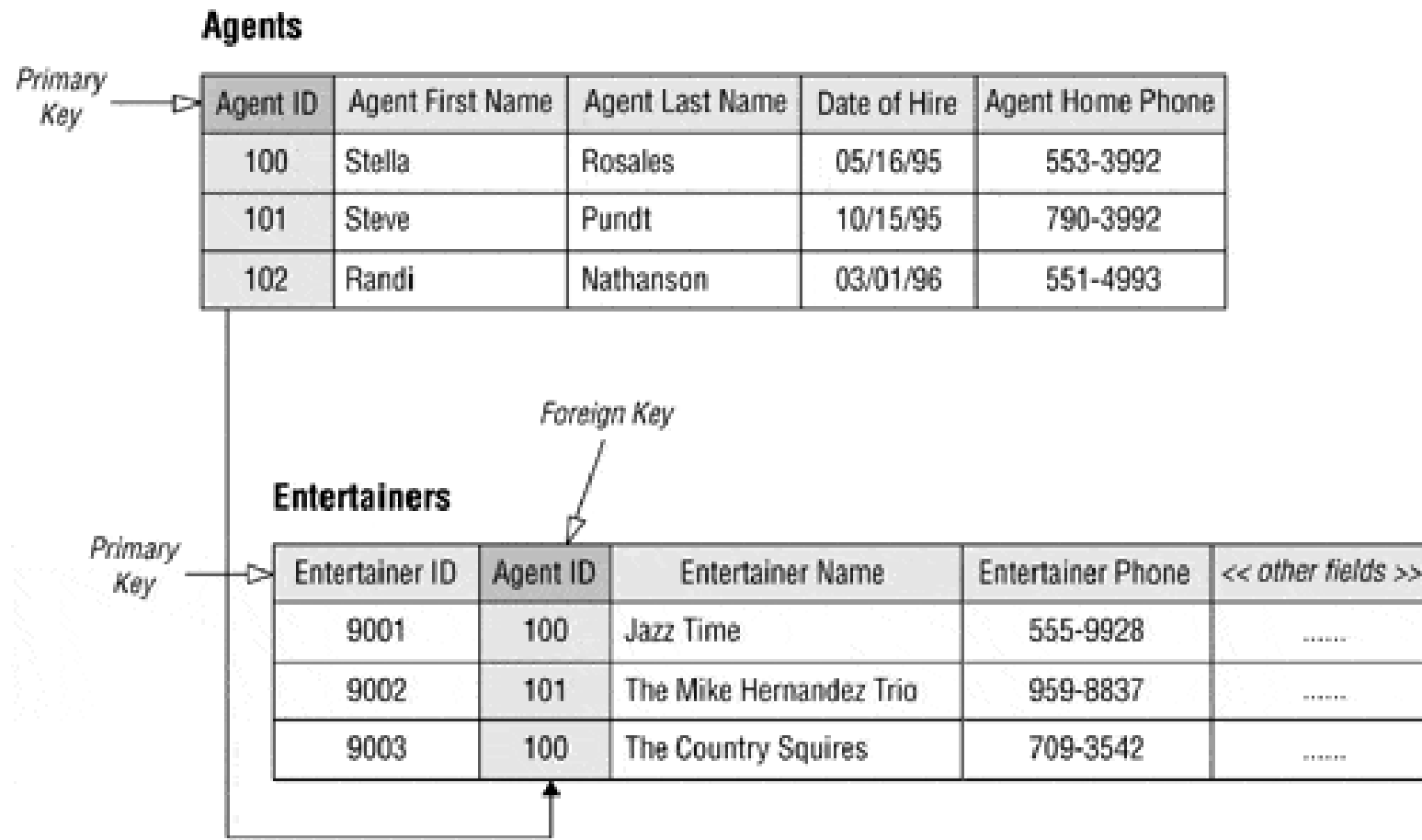
Customer ID	Tax ID	Name	Address	[More fields...]
1234567890	555-5512222	Munmun	323 BroadWay	...
2223344556	555-5523232	Wile E.	1200 Main Street	...
3334445563	555-5533323	Ekta	871 1st Street	...
4232342432	555-5325523	E. F. Codd	123 It Way	...

Update/Write causes row locking

Relational Databases

Relational Model

- The relational model can have a number of keys to identify records, these include:
 - Primary Keys (PK)
 - Composite Keys (CK)
 - Foreign Keys (FK)



Relational Databases

Relational Model

- Primary Keys
 - A field offering a unique identifier
 - Used to identify a unique tuple
 - May be an auto-incrementing numeric value

Store ID	Purchase Location
1	Los Angeles
2	New York
3	San Francisco

Relational Databases

Relational Model

- Composite Keys
 - Used to identify a unique tuple
 - A combination of two or more columns
 - May employ auto incrementing on one, some or all columns involved

Customer ID	Store ID	Purchase Location
1	1	Los Angeles
1	3	San Francisco
2	1	Los Angeles
3	2	New York
4	3	San Francisco

Relational Databases

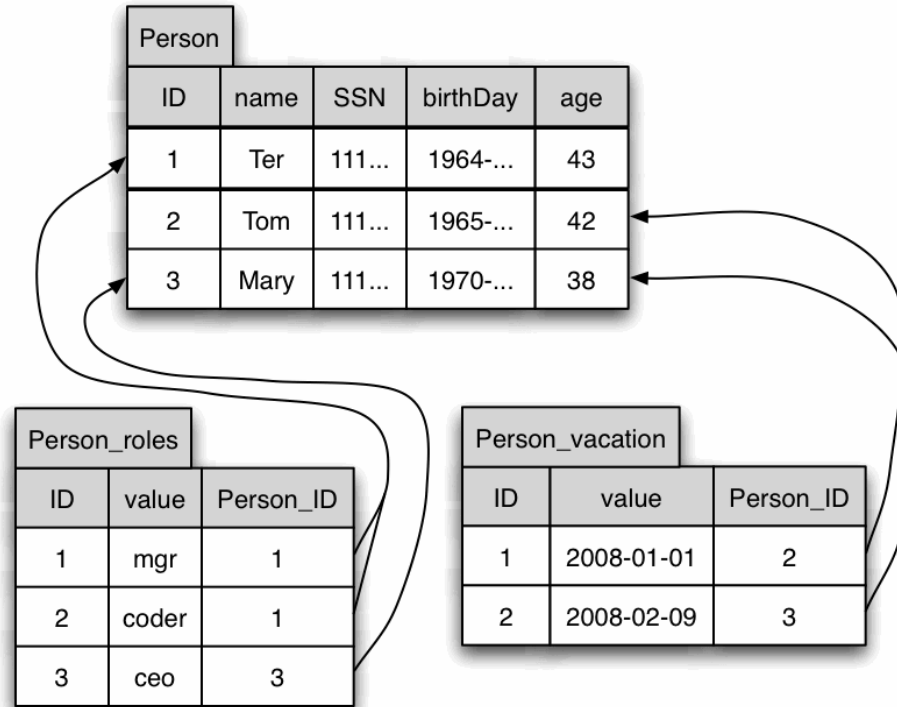
Relational Model

- Foreign Keys
 - A field in one table that uniquely identifies a key in another table
 - Facilitates selection of records across tables
 - Enforces referential integrity
 - May be an auto-incrementing numeric value

Relational Databases

Relational Model

- Foreign Keys



Relational Databases

Normalization

- Relations/Tables should be carefully defined
- The databases should be “Normalized”
- Normalization is the process of reorganising attributes into tables with the goal of minimising redundancy
 - Increases storage efficiency
 - May have performance implications
- Normalization may take a number of forms

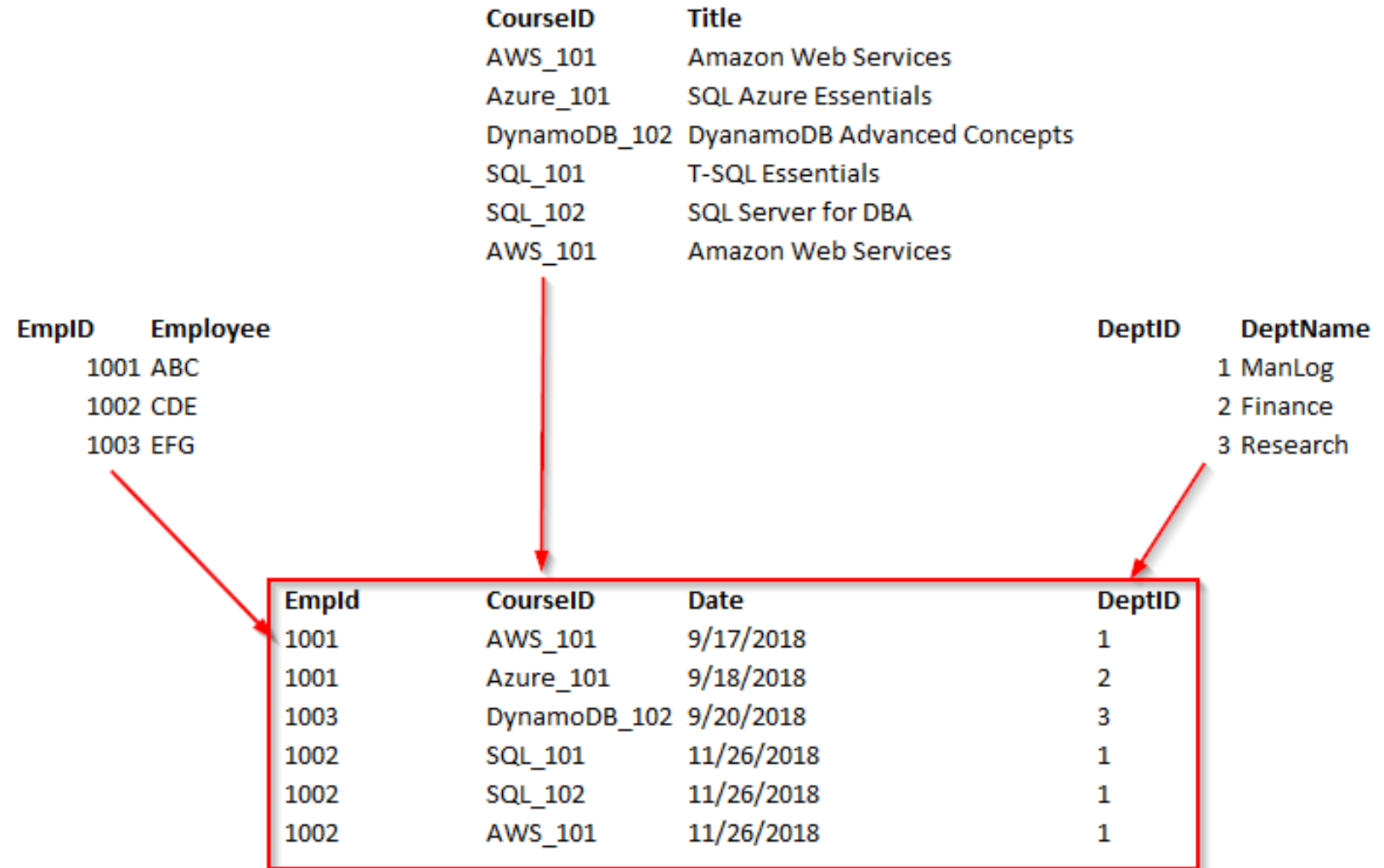
Relational Databases

Normalization

- An example of normalization in a sale database
 - Storing sales records and product records in separate tables
 - Product records are stored within unique IDs
 - Records in the sales table reference items sold through referencing a unique ID

Relational Databases

Normalization



Relational Databases

Normalization

- Three different levels of normalization
 - 1NF – First Normal Form
 - 2NF – Second Normal Form
 - 3NF – Third Normal Form
- A DB is described as normalized when it satisfies the 3NF

Relational Databases

Normalization – 1NF

- 1NF enforces the following criteria
 - Eliminate repetition in individual tables
 - Create a separate table for each set of related data/entity type
 - Identify each record with a primary key
 - Contains only atomic values

TABLE_PRODUCT

Product ID	Color	Price
1	red, green	15.99
2	yellow	23.99
3	green	17.50
4	yellow, blue	9.99
5	red	29.99

Relational Databases

Normalization – 1NF

- Given the requirement to store multiple phone numbers
- A violates the requirement as telephone number may only contain one atomic value
- B satisfies the 1NF but not more complex forms

Customer ID	First Name	Surname	Telephone Number
123	Robert	Ingram	555-861-2025
456	Jane	Wright	555-403-1659 555-776-4100
789	Maria	Fernandez	555-808-9633

A

Customer ID	First Name	Surname	Telephone Number
123	Robert	Ingram	555-861-2025
456	Jane	Wright	555-403-1659
456	Jane	Wright	555-776-4100
789	Maria	Fernandez	555-808-9633

B

Relational Databases

Normalization – 2NF

- 2NF enforces the following criteria
 - Satisfy criteria expressed in 1NF
 - Non-key attributes are fully functionally dependent on the primary key

Relational Databases

Normalization – 2NF

- A presents a table using a Key that is not normalised.
- A violates the requirement as Purchase Location only depends on store ID which is only an element of the PK
- B satisfies the 2NF

Customer ID	Store ID	Purchase Location
1	1	Los Angeles
1	3	San Francisco
2	1	Los Angeles
3	2	New York
4	3	San Francisco

A

Customer ID	Store ID
1	1
1	3
2	1
3	2
4	3

Store ID	Purchase Location
1	Los Angeles
2	New York
3	San Francisco

B

Relational Databases

Normalization – 3NF

- 3NF enforces the following criteria
 - Satisfy criteria expressed in 2NF
 - “Every non-prime attribute of R is non-transitively dependent on every key of R”
- More simply put:
“Every non-key attribute must provide a fact about the key, the whole key, and nothing but the key.” – Bill Kent

Relational Databases

Normalization – 3NF

- A is not in the 3NF, Book ID determines Genre ID and Genre ID determines Genre Type. This represents a transitive functional dependency.

Book ID	Genre ID	Genre Type	Price
1	1	Gardening	25.99
2	2	Sports	14.99
3	1	Gardening	10.00
4	3	Travel	12.99
5	2	Sports	17.99

A

- B is brought to the 3rd normal form by splitting into two tables

Book ID	Genre ID	Price
1	1	25.99
2	2	14.99
3	1	10.00
4	3	12.99
5	2	17.99

Genre ID	Genre Type
1	Gardening
2	Sports
3	Travel

B

Relational Databases

Entity-Relationship Modelling

- Mapping between tables may take a number of forms:
 - 1 to 1 – e.g one person may have one social security number
 - 1 to Many - e.g one person may have many bank accounts
 - Many to Many – e.g. books may have many authors, and vice versa

Book ID	Genre ID	Price
1	1	25.99
2	2	14.99
3	1	10.00
4	3	12.99
5	2	17.99

Genre ID	Genre Type
1	Gardening
2	Sports
3	Travel

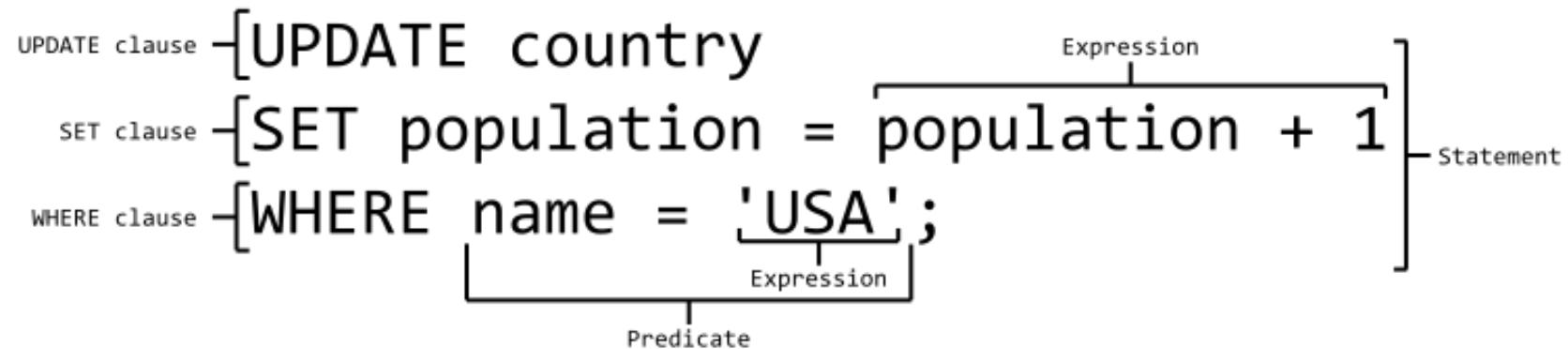
Relational Databases

SQL

- SQL is the primary mechanism for interacting with RDBMS
- SQL is subdivided into a number of language elements, including:
 - Clauses - components of statements and queries
 - Expressions – used to evaluate a value
 - Predicates – conditions to be evaluated within SQL
 - Queries – Retrieve data based on criteria
 - Statements – instructions which may have an effect on data or DB schema

Relational Databases

SQL



Update statement

Country is the table

Population is the field

The expression indicates the current field + 1

Where the name is equal to the US

Relational Databases

SQL

- Examples of clauses include:
 - SELECT – select data from a database
 - INSERT – insert data into a database
 - UPDATE – update records in a database
 - DELETE – delete records in a database
 - WHERE – used to restrict the operation of a statement
 - LIMIT – used to limit the number of records returned by a resultset

Relational Databases

SQL

- Examples of clauses include
 - JOIN – used to combine rows from 2+ tables
 - DISTINCT – used to remove duplicates from the result set produced by a select statement
 - ORDER BY – used to order the result set of a query by a specified column list
 - GROUP BY – used in a select statement to collect data across multiple records and group the results

Relational Databases

SQL

- Examples of clauses include
 - HAVING – similar to where but applied to aggregate functions, such as GROUP BY
 - UNION - combine results of 2+ select statements without returning duplicate rows

Relational Databases

SQL

- The following is the structure for a statement that creates a table within SQL

```
CREATE TABLE table_name  
(  
  column_name1 data_type(size),  
  column_name2 data_type(size),  
  column_name3 data_type(size),  
  PRIMARY KEY (column_name)  
);
```

Relational Databases

SQL – Data types

- SQL supports a number of data types including:
 - CHAR – a number of characters up to 255; char (30)
 - VARCHAR – a number of characters up to 65,535; varchar (300)
 - TEXT – strings/text cannot be ordered or used in a where clause, in most cases
 - BINARY – binary values
 - BLOB – binary strings

Relational Databases

SQL – Data types

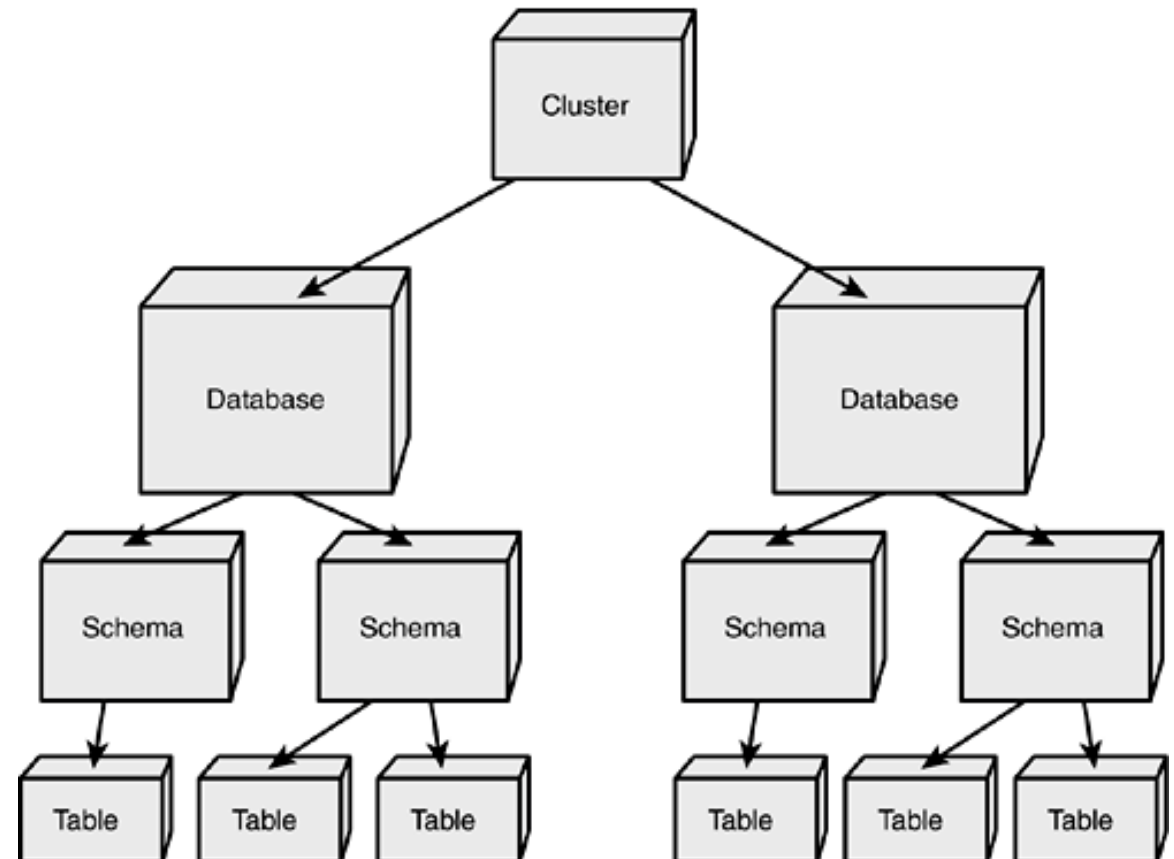
- SQL supports a number of data types including:
 - INT – whole numbers, 4 bytes, signed or unsigned
 - -2147483648 to 2147483647
 - 0 to 4294967295
 - BIGINT – whole numbers, 8 bytes, signed or unsigned
 - -9223372036854775808 to 9223372036854775807
 - 0 to 18446744073709551615
- These may vary with RDBMS implementation
- Correct selection and specification is imperative
- Incorrect specification has performance implications

Relational Databases

Storage hierarchy

There is a storage hierarchy present within Relational databases:

1. Database server
2. Schemas/Databases*
3. Tables
4. Relations



Relational Databases

Advanced topics

Join – This allows querying across tables to produce a record set/Revlar

Consider the tasks table coexisting with the following “task_operator” table

```
CREATE TABLE `example_schema`.`task_operator` ( `operator_id` INT NOT NULL AUTO_INCREMENT, `task_id` INT NULL, `operator_name` VARCHAR(450) NULL, PRIMARY KEY (`operator_id`));
```

Column	Type	Default Value	Nullable
operator_id	int(11)		NO
task_id	int(11)		YES
operator_name	varchar(450)		YES

Relational Databases

Advanced topics

The task operator table contains the following

	operator_id	task_id	operator_name
	1	1	Anon
	2	2	Bnon
	3	3	Cnon
	4	1	Dnon
▶*		NULL	NULL

Relational Databases

Advanced topics

Using join it is possible to find the operators associated with a task simply by specifying the task name.

```
SELECT T1.operator_name FROM example_schema.task_operator T1  
INNER JOIN example_schema.tasks T2 ON T2.task_id = T1.task_id  
WHERE T2.subject = "SubjectName";
```

	operator_name
	Anon
	Dnon



INNER JOIN Returns records that have matching values in both tables

NoSQL

Semi Structured & Unstructured data



Semi-structured data **is less organized** than structured data, and is not stored in a relational format, as **the fields do not neatly fit into tables, rows, and columns**.

Semi-structured data contains tags that make the organization and hierarchy of the data apparent - for example, key/value pairs.

The expression and structure of the data in this style is defined by a **serialization language/document format**.

For software developers, data serialization languages are important because they can be used to write data stored in memory to a file, sent to another system, parsed and read.

There are three common formats for semi-structured data

```
<Person Age="23">
  <FirstName>John</FirstName>
  <LastName>Smith</LastName>
  <Hobbies>
    <Hobby Type="Sports">Golf</Hobby>
    <Hobby Type="Leisure">Reading</Hobby>
    <Hobby Type="Leisure">Guitar</Hobby>
  </Hobbies>
</Person>
```

```
{
  "firstName": "John",
  "lastName": "Doe",
  "age": "23",
  "hobbies": [
    { "type": "Sports", "value": "Golf" },
    { "type": "Leisure", "value": "Reading" },
    { "type": "Leisure", "value": "Guitar" }
  ]
}
```

```
firstName: John
lastName: Doe
age: 23
hobbies:
  - type: Sports
    value: Golf
  - type: Leisure
    value: Reading
  - type: Leisure
    value: Guitar
```

XML

JSON

YAML

Unstructured Data

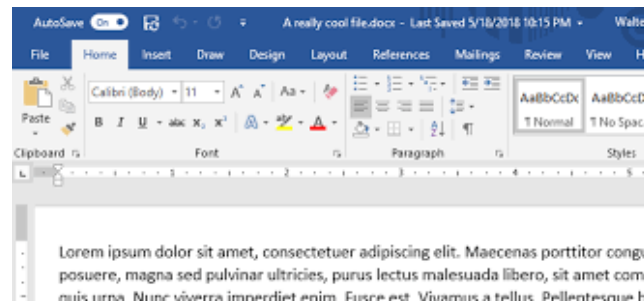
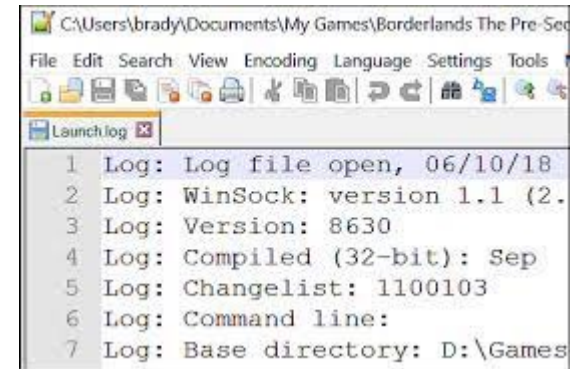
The organization of **unstructured data** is ambiguous.

Unstructured data is often delivered in **files**, such as photos or videos.

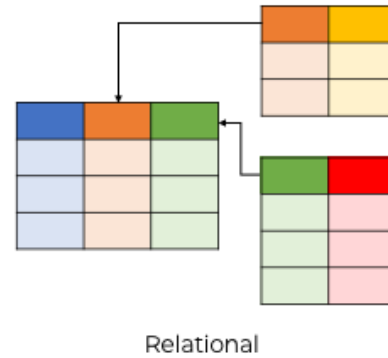
A video file itself may have an overall structure and come with semi-structured metadata, but the **data that comprises the video itself is unstructured**. Therefore, **photos, videos, and other similar files are classified as unstructured data**.

Examples of unstructured data include:

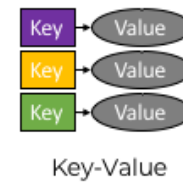
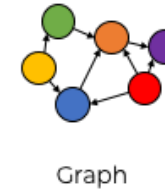
- Media files, such as photos, videos, and audio files
- Office files, such as Word documents
- Text files
- Log files



SQL DATABASES



NoSQL DATABASES



NoSQL



Gaming



Social



IoT



Web



Mobile



Enterprise



Key/value store



Document database



Column family store

SQL



Web



Mobile



Enterprise



Data mart



Relational table storage

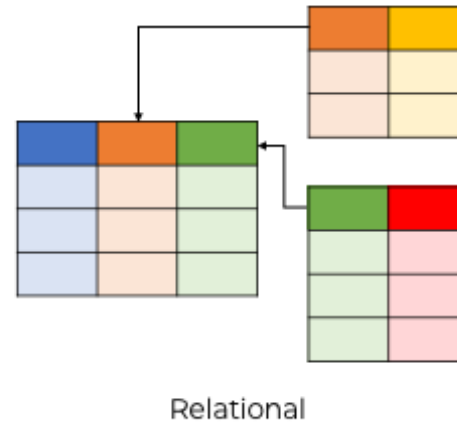


Relationships use joins

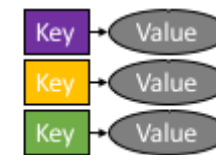
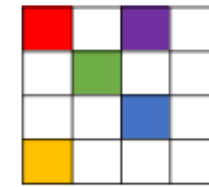
NoSQL

- NoSQL is an umbrella term for a newer generation of databases where data is modelled without use of traditional 'relations'
- Typically support SQL-Like query languages, a more accurate term would be NoREL query
- Data may be modelled in a number of forms

SQL DATABASES



NoSQL DATABASES



- NoSQL DBs typically have advantages and disadvantages over relational DBs
- Relational databases have a number of problems reducing their applicability to solutions
 - *Additional overheads from Rules and Axioms to give relational functionality may reduce performance*
 - *Difficult to scale across nodes*
 - *Use of an inflexible data structure*

Aspect	NoSQL	RDBMS
Availability	Continuous	Good
Consistency	Poor	Good
Data handling	Supports big data scale workloads	Medium to large workloads*
Performance	High	Low
Reliability	Low to Moderate*	Good
Read/Write	Scaling Reads and Writes	Scaling Reads
Scalability	High	Moderate**

Examples of NoSQL Databases include:

1. *Columnar Databases* – Such as *Cassandra*
2. *Key-Value Data bases* – Such as *Redis*
3. *Document Databases* – Such as *Mongo*
4. *Graph Databases* – Such as *Neo4J*



mongoDB®



redis

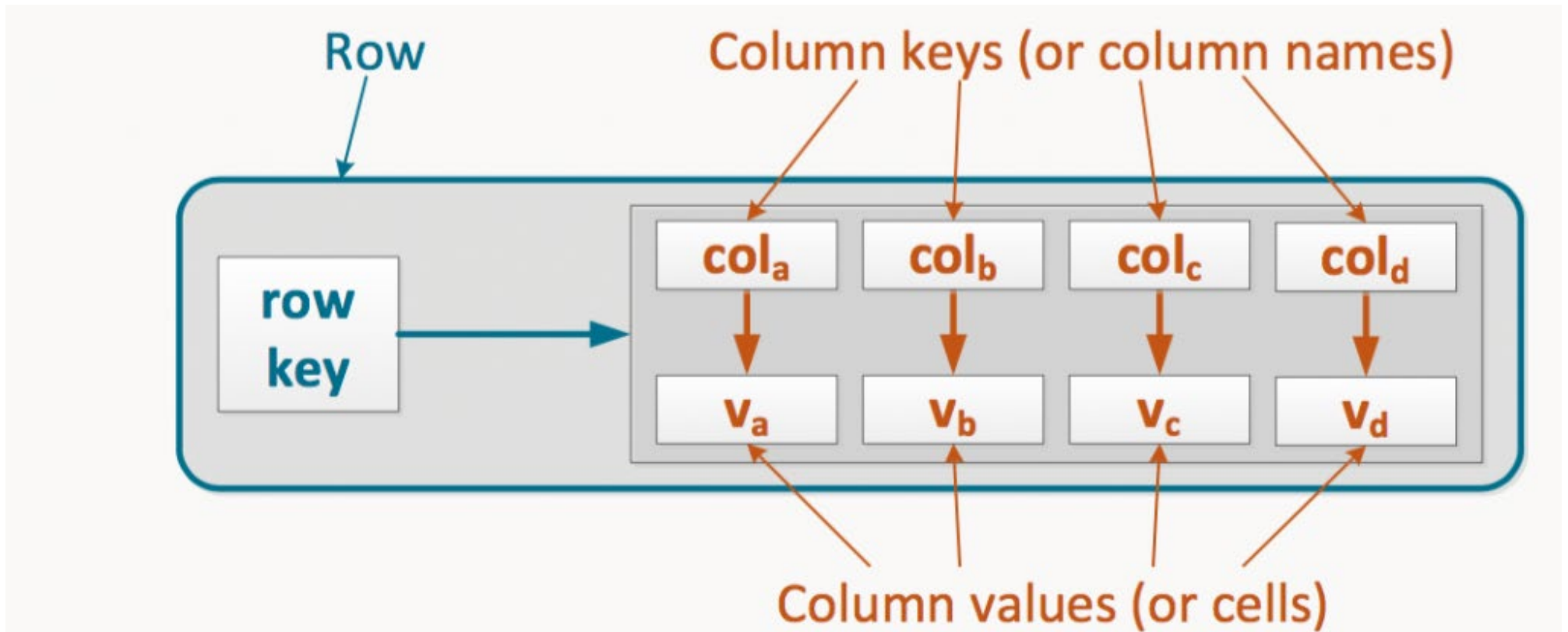
1. Columnar Databases

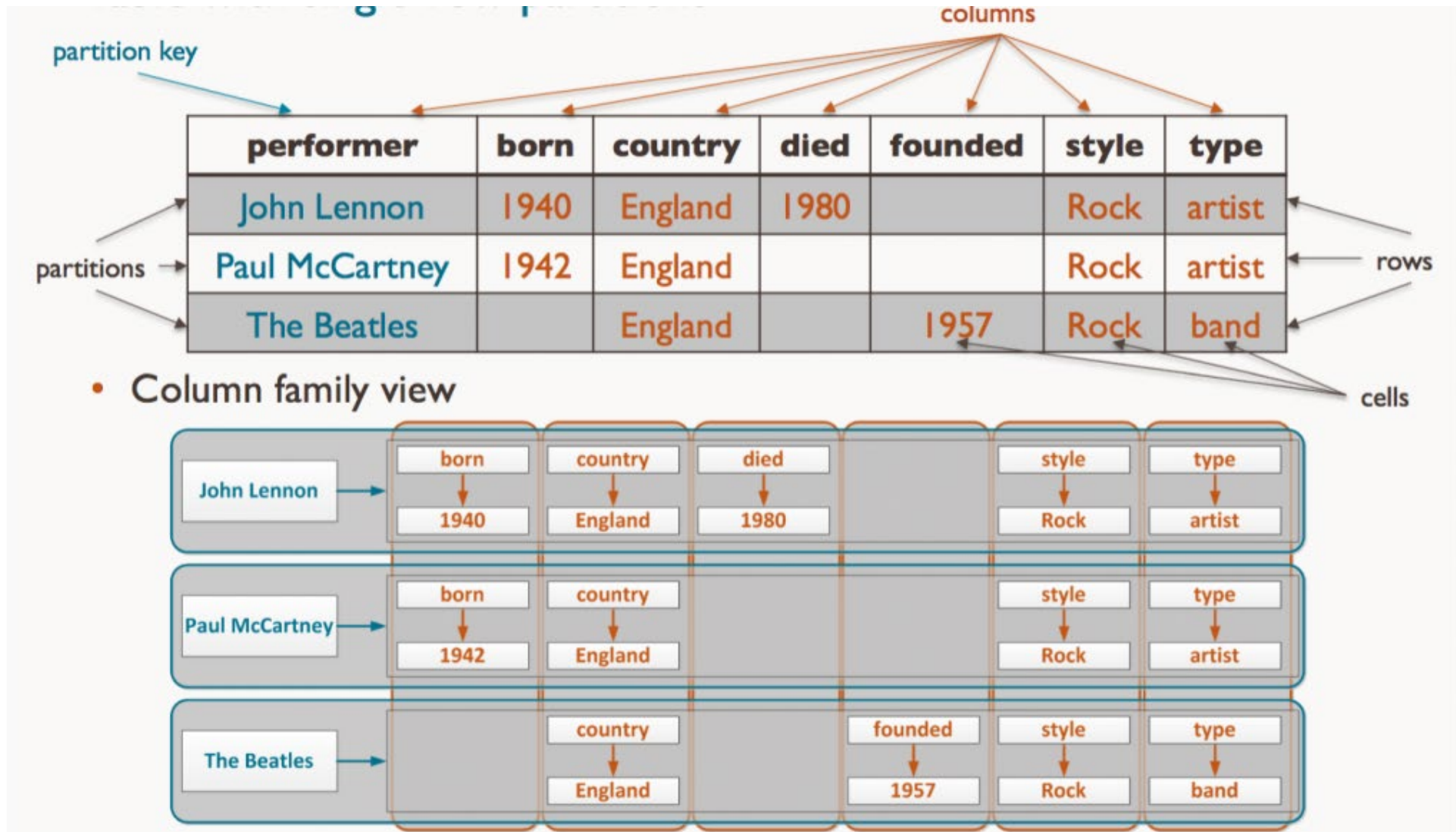
- Columnar databases organize data in rows and columns, but **with a twist** from the traditional RDBMS.
- Also known as wide-column stores or partitioned row stores, they provide the option to **organize related rows in partitions that are stored together on the same replicas to allow fast queries.**
- Unlike RDBMSs, the **Columnar format is not necessarily strict.** For example, Apache Cassandra does not require all rows to contain values for all columns in the table.
- Columnar databases use **hashing to retrieve rows** from the table.



Columnar Databases

Here's what an individual row within a column family looks like in Cassandra:





Use Cases

Spotify uses Cassandra to store user profile attributes and metadata about artists, songs, etc. for better personalization



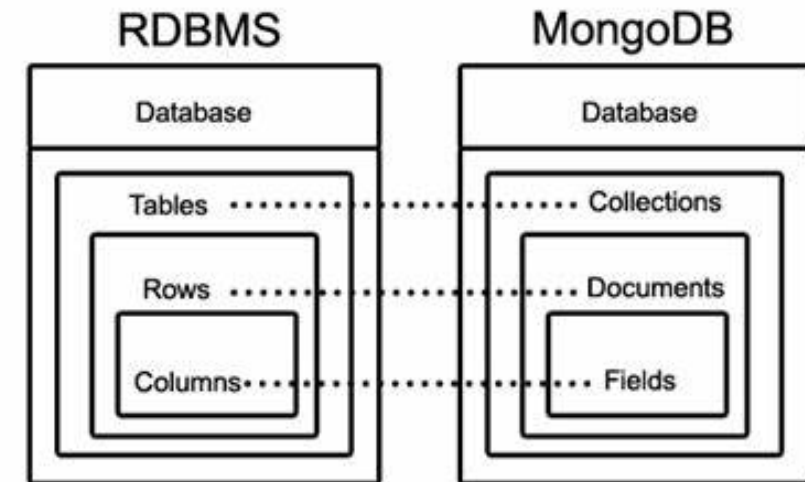
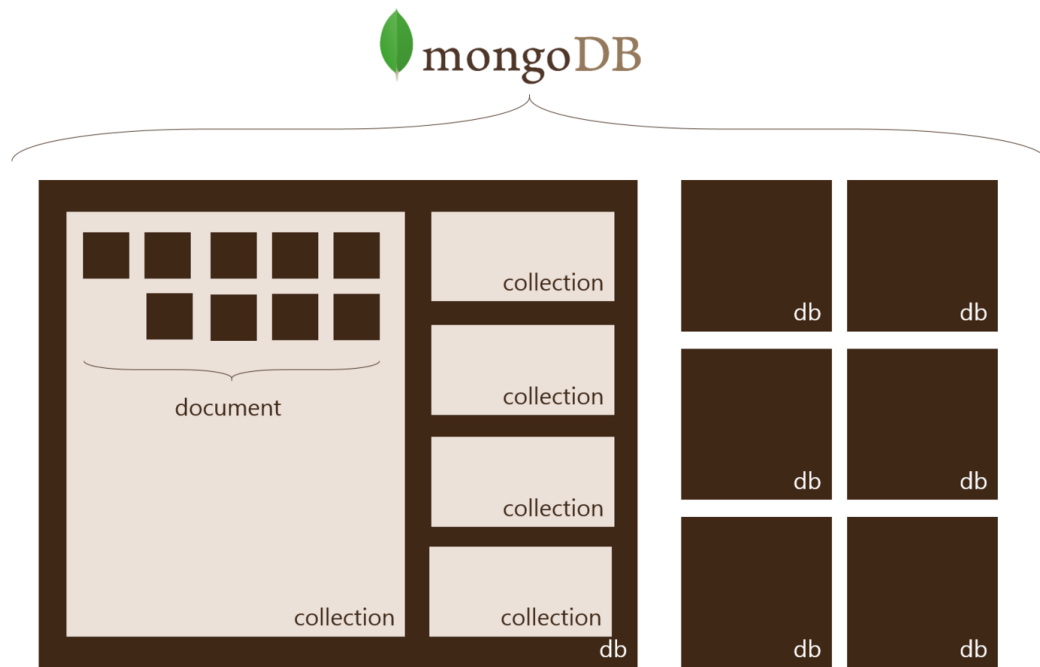
Facebook initially built its revamped Messages on top of **HBase** but is now also used for other Facebook services like the Nearby Friends feature and search indexing



Outbrain uses Cassandra to serve over 190 billion personalized content recommendations each month

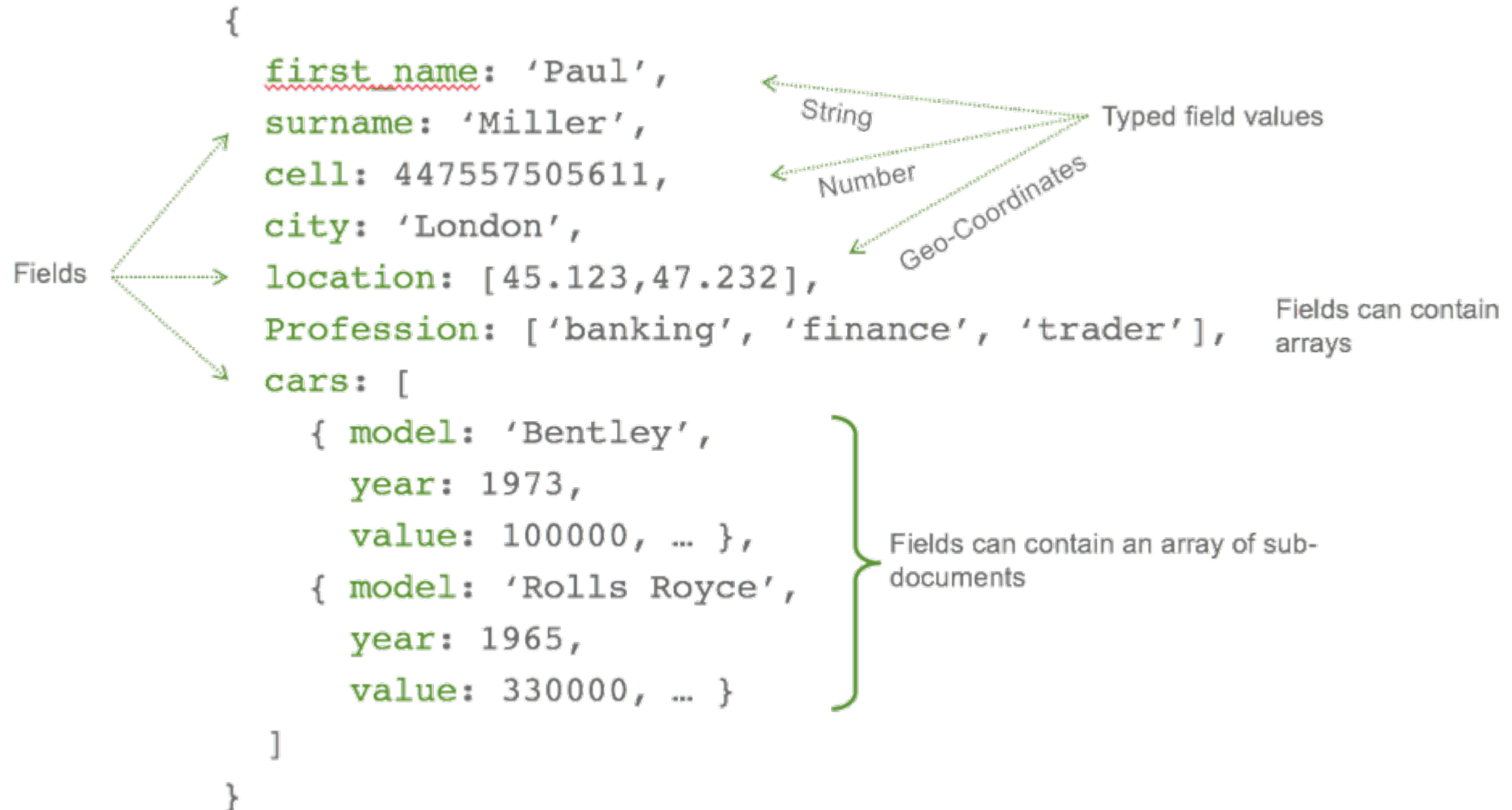


Document Databases



- Document databases expand on the basic idea of key-value stores where “documents” are more complex, in that they contain data and **each document is assigned a unique key**, which is used to retrieve the document.
- These are designed for storing, retrieving, and **managing document-oriented information, often stored as JSON**. Each document can contain different types of data. **Groups of documents are called collections**. Each document in a collection can have a different structure.
- Since the document database can inspect the document contents, the database can perform some additional retrieval processing. Unlike RDBMSs which require a static schema, **Document databases have a flexible schema as defined by the document contents**.





Use Cases

- SEGA uses MongoDB for handling 11 million in-game accounts
- Cisco moved its VSRM (video session and research manager) platform to Couchbase to achieve greater scalability
- Aer Lingus uses MongoDB with Studio 3T to handle ticketing and internal apps
- Built on MongoDB, The Weather Channel's iOS and Android apps deliver weather alerts to 40 million users in real-time



Due to the multiple API access model there are several ways to query documents in CosmosDB

- Consider a collection called families containing the following JSON document:

JSON

```
{
  "id": "WakefieldFamily",
  "parents": [
    { "familyName": "Wakefield", "givenName": "Robin" },
    { "familyName": "Miller", "givenName": "Ben" }
  ],
  "children": [
    {
      "familyName": "Merriam",
      "givenName": "Jesse",
      "gender": "female", "grade": 1,
      "pets": [
        { "givenName": "Goofy" },
        { "givenName": "Shadow" }
      ]
    },
    {
      "familyName": "Miller",
      "givenName": "Lisa",
      "gender": "female",
      "grade": 8 }
  ],
  "address": { "state": "NY", "county": "Manhattan", "city": "NY" },
  "creationDate": 1431620462,
  "isRegistered": false
}
```

It would be possible to use the SQL interface to select documents that match that families ID:

SQL

```
SELECT *  
FROM Families f  
WHERE f.id = "WakefieldFamily"
```

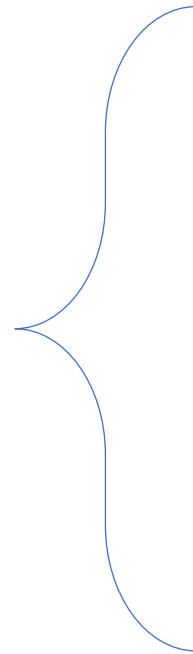
JSON

```
{  
  "id": "WakefieldFamily",  
  "parents": [  
    { "familyName": "Wakefield", "givenName": "Robin" },  
    { "familyName": "Miller", "givenName": "Ben" }  
  ],  
  "children": [  
    {  
      "familyName": "Merriam",  
      "givenName": "Jesse",  
      "gender": "female", "grade": 1,  
      "pets": [  
        { "givenName": "Goofy" },  
        { "givenName": "Shadow" }  
      ]  
    },  
    {  
      "familyName": "Miller",  
      "givenName": "Lisa",  
      "gender": "female",  
      "grade": 8 }  
  ],  
  "address": { "state": "NY", "county": "Manhattan", "city": "NY" },  
  "creationDate": 1431620462,  
  "isRegistered": false  
}
```


It would be possible to use the SQL interface to extract the given names for children in a family whose ID matches 'WakefieldFamily':

SQL

```
SELECT c.givenName  
FROM Families f  
JOIN c IN f.children  
WHERE f.id = 'WakefieldFamily'
```



```
[  
  {  
    "givenName": "Jesse"  
  },  
  {  
    "givenName": "Lisa"  
  }  
]
```

It would be possible to use the SQL interface to extract the given names for children in a family whose ID matches 'WakefieldFamily':

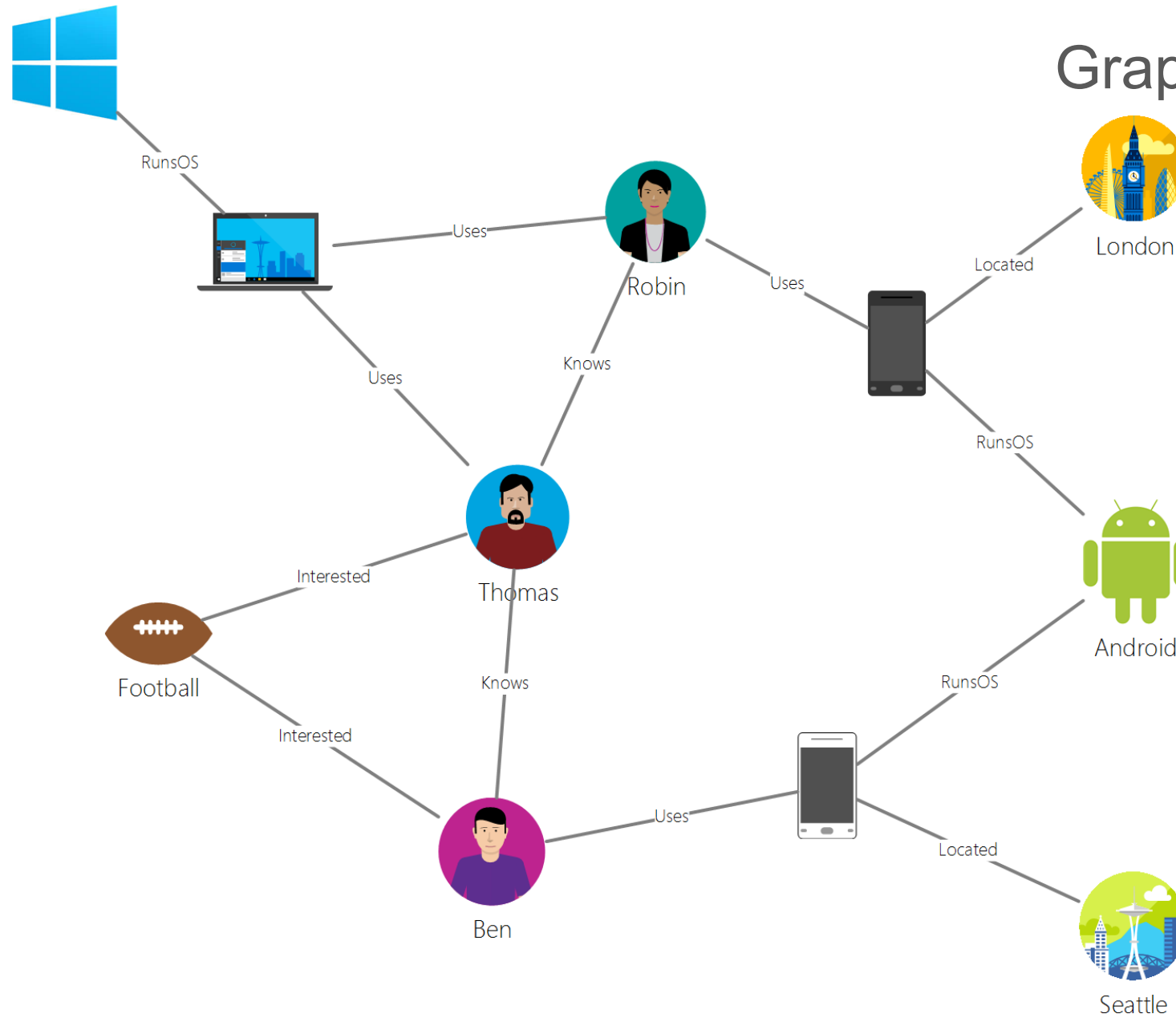
Bash

```
db.families.find( { id: "WakefieldFamily" }, { children: true } )
```

JSON

```
{
  "_id": "ObjectId("58f65e1198f3a12c7090e68c")",
  "children": [
    {
      "familyName": "Merriam",
      "givenName": "Jesse",
      "gender": "female",
      "grade": 1,
      "pets": [
        { "givenName": "Goofy" },
        { "givenName": "Shadow" }
      ]
    },
    {
      "familyName": "Miller",
      "givenName": "Lisa",
      "gender": "female",
      "grade": 8
    }
  ]
}
```

Graph Databases



Graph databases **store their data using a graph metaphor** to exploit the relationships between data.

Nodes in the graph represent data items, and edges represent the relationships between the data items.

Graph databases are **designed for highly complex and connected data**, which outpaces the relationship and join capabilities of an RDBMS.

Graph databases are often exceptionally **good at finding commonalities and anomalies** among large data sets.



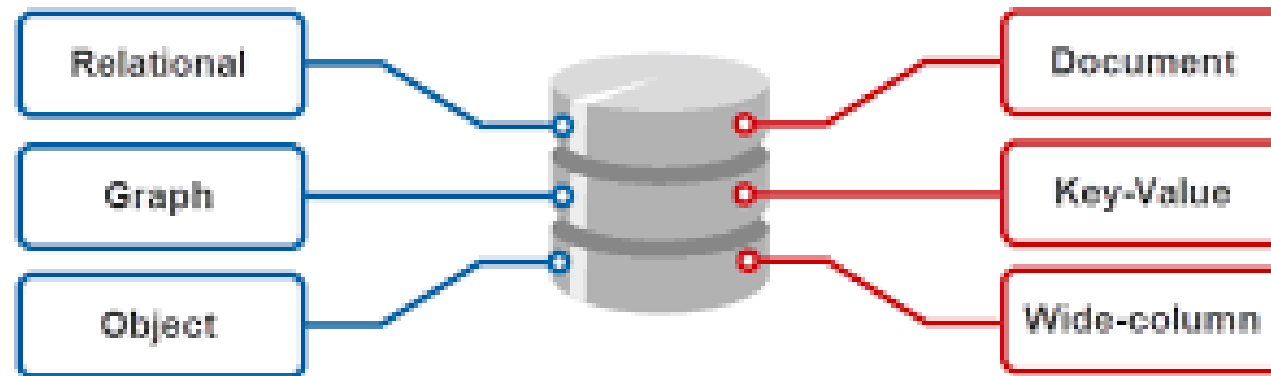
Use Cases

Graph databases are great for establishing data relationships especially when dealing with large data sets. They offer blazing fast query performance that relational databases cannot compete with, especially as data grows much deeper.

- **Walmart** use Neo4j to provide customers personalized, relevant product recommendations and promotions
- **Medium** use Neo4j to build their social graph to enhance content personalization
- **Cisco** use Neo4j to mine customer support cases to anticipate bugs
- **Tibco** use graph databases to provide fraud detection solutions



Multi-Model Databases

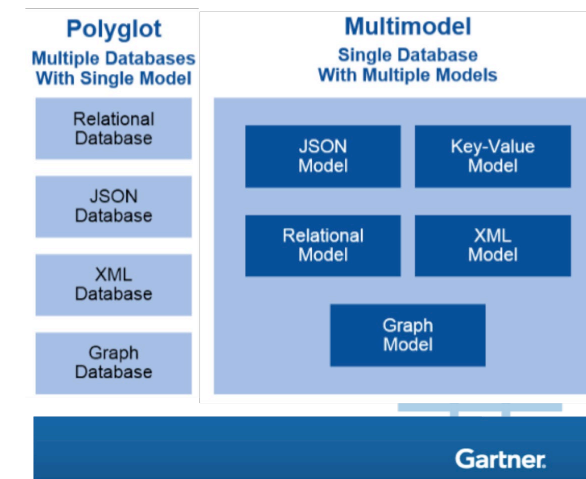


Multi-Model Databases

Multi-model databases are an emerging trend in both the NoSQL and RDBMS markets.

They are designed to support multiple data models against a single, integrated backend. Most database management systems are organized around a single data model that determines how data can be organized, stored, and manipulated.

By contrast, a multi-model database allows an enterprise to store parts of the system's data in different data models, simplifying application development.

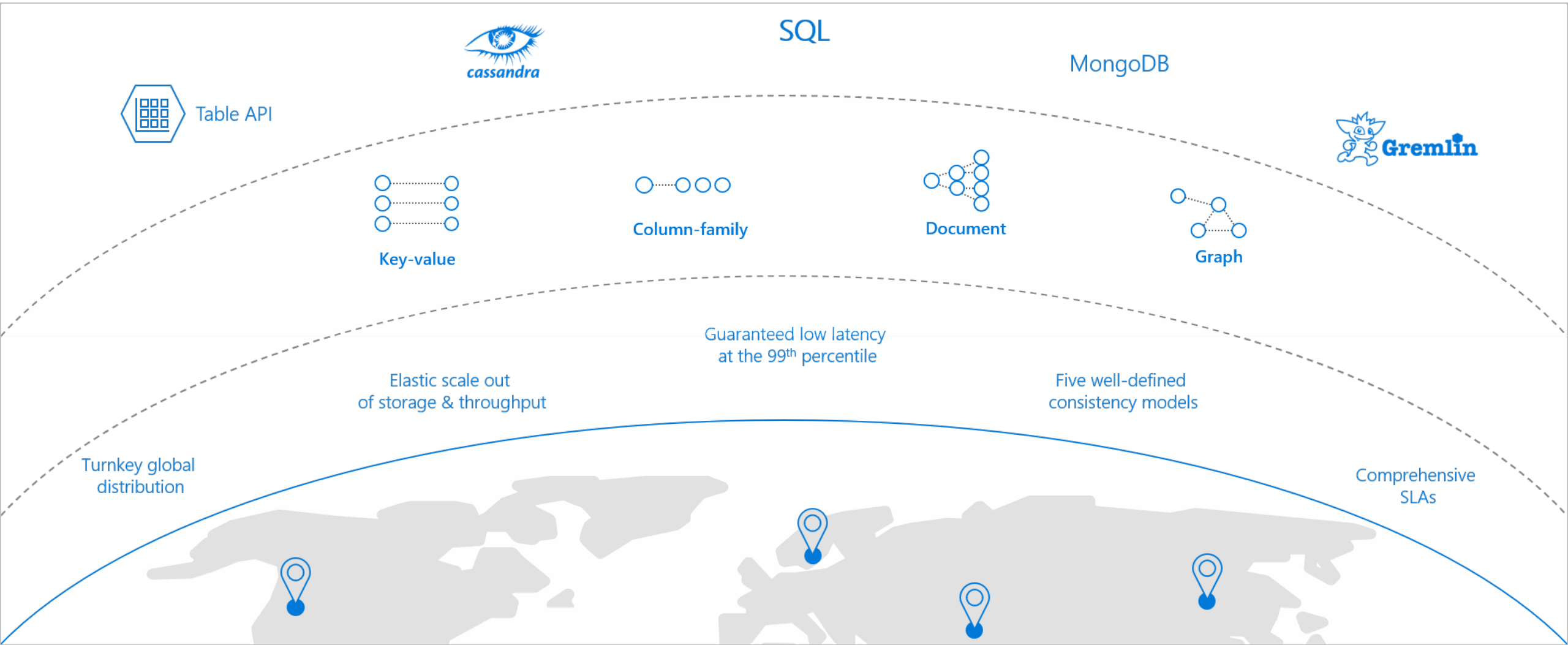


Use Cases

- **RTE's Investigations Unit** uses OrientDB to establish data connections across various sources for investigative purposes
- Azure CosmosDB supports **Citrix's Identity Platform**, which enables single-sign on for more than 400,000 organizations and 100 million individuals worldwide
- **ASOS** uses Azure Cosmos DB to deliver real-time product recommendations and instant order updates for 15.4 million customers
- **Ulster University** use a multimodel sensor data platform for the storage and processing of billions of diverse sensor data elements



Multi-Model - CosmosDB



- Azure Cosmos DB is a **fully managed**, **multimodel**, NoSQL database for development.
- **Single-digit millisecond response times**, and **automatic** and **instant scalability**.
- As a fully managed service, Azure Cosmos DB takes database administration off your hands with automatic management, updates and patching.
- It also handles capacity management with **cost-effective**, **serverless** and **automatic scaling** options that respond to application needs to match capacity with demand.
- It supports open source compatible APIs and SDKs for popular languages.

Azure Cosmos DB

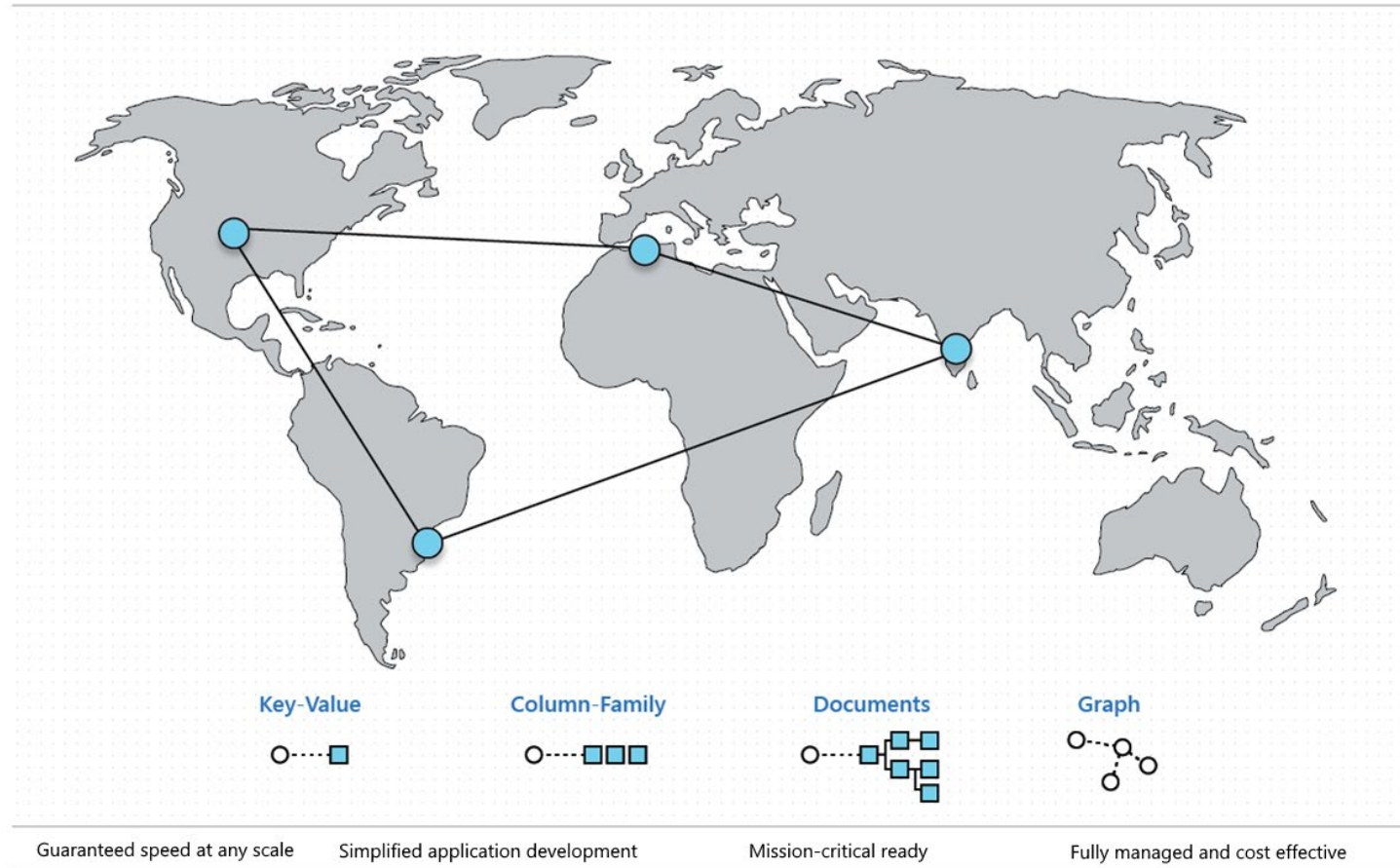
SQL
SQL

{LEAF}
API for MongoDB


Gremlin


Cassandra


Table



Azure SQL Deployments

Within the umbrella of the Azure SQL platform, there are **many deployment options and choices** that you need to make to meet your needs.

These options give you the **flexibility to get and pay for exactly what you need.**

The deployment options include:

- **SQL Server on virtual machines**
- **Azure SQL Managed Instance**
- **Azure SQL Database**
- **Elastic database pools**

Azure SQL Deployments

SQL virtual machines



Best for migrations and applications requiring OS-level access

Managed instances



Best for most lift-and-shift migrations to the cloud

Databases



Best for modern cloud applications. Hyperscale and serverless options are available

Azure SQL Deployments

SQL Server on Azure VMs provides the promise of the cloud while maintaining OS control



Customer challenge

I want to migrate to the cloud as fast as possible but maintain operating system control and complete SQL Server functionality



Solution

Get the combined performance, security, and analytics of SQL Server, backed by the flexibility, security, and hybrid connectivity of Azure

Key features

- SQL Server and OS server access
- Expansive SQL and OS versions
- Windows, Linux, Containers
- File stream, DTC, and Simple Recovery model
- SSAS, SSRS, and SSIS

Azure differentiators

- Free Extended Security Updates for SQL Server 2008/R2
- Automated backups and security updates
- Point-in-time restore with Azure backup
- Accelerated storage performance with Azure Blob Caching
- 435 percent overall return on an Azure IaaS investment over five years¹

Azure virtual machine



Healthcare software manufacturer saves costs when reusing licenses while moving 600 on-premises VMs to Azure

1. Forrester Consulting. The Total Economic Impact™ of Microsoft Azure SQL Database Managed Instance.

Azure SQL Deployments

SQL Server on Azure Virtual Machines

SQL Server on a virtual machine is a version of SQL Server that runs in an Azure VM.

It's SQL Server, so all your skills should directly transfer, though Azure can help automate backups and security patches.

Consumers are responsible for updating and patching the OS and SQL Server, apart from critical SQL Server security patches.

Azure SQL Deployments



Customer challenge

I want to migrate to the cloud, remove management overhead, but I need instance-scoped features. (Service Broker, SQL Server Agent, CLR...)

Azure SQL managed instance eases cloud migration



Solution

Managed instance combines leading security features with SQL Server compatibility and business model designed for on-premises customers

Key features

- Single instance or instance pool
- SQL Server surface area (vast majority)
- Native virtual network support
- Fully managed service
- On-premises identities enabled with Azure AD and AD Connect

Azure differentiators

- Near zero downtime migration using log shipping
- Fully managed business continuity with failover groups
- Projected return on investment of 212 percent over three years¹
- The best of SQL Server with the benefits of a managed service

Azure SQL managed instance



Komatsu easily migrated 1.5 TBs of data thanks to near complete compatibility with SQL Server, plus 49% cost reduction and 25-30% performance gains.

1. Forrester Consulting. The Total Economic Impact™ of Microsoft Azure SQL Database Managed Instance.

Azure SQL Deployments

SQL Managed Instance

SQL Managed Instance is a **PaaS deployment option of Azure SQL**.

It gives you an instance of SQL Server but removes much of the overhead of managing a virtual machine.

This option is ideal for customers who want to use instance-scoped features and want to move to the cloud without rearchitecting their applications.

Instance-scoped features are features that are tied to an instance of SQL Server, such as **Machine Learning Services**, as opposed to features that are tied to a database in an instance of SQL Server.

SQL Managed Instance allows you to access instance-scoped features. But you don't have to worry about, nor do you have access to, the OS or the infrastructure underneath.

Azure SQL Deployments



Customer challenge

I want to build modern apps, potentially multi-tenanted, with the highest uptime and predictable performance

Azure SQL database is built for modern cloud apps



Solution

Azure SQL database is a highly-scalable cloud database service with built-in high availability and machine learning

Key features

- Single database or elastic pool
- Hyperscale storage (100TB+)
- Serverless compute
- Fully managed service
- Private link support
- High availability with AZ isolation

Azure differentiators

- Industry highest availability SLA of 99.995%
- Industry only business continuity SLA with 5 second RPO and 30 second RTO
- Price-performance leader for mission-critical workloads while costing up to 86 percent less than AWS RDS (GigaOm)

Azure SQL database



AccuWeather uses Azure SQL database to provide an automated, scalable weather prediction service.

Azure SQL Deployments

Azure SQL Database

SQL Database is a PaaS deployment option of Azure SQL that abstracts both the OS and the SQL Server instance away from users.

This deployment option allows you to just get a database and start developing applications.

SQL Database is also the only deployment option that supports scenarios that require unlimited database storage (hyperscale) and autoscaling for unpredictable workloads (serverless).

SQL Database has the industry's highest availability SLA. It provides other intelligent capabilities related to monitoring and performance, partly because Microsoft manages instances.

Azure SQL Deployments

Elastic database pool

Elastic database pools allow you to **share resources among multiple instances** and databases and optimize your costs.

SQL Database elastic pools allow you to **host many databases within a single set of provisioned SQL Database resources**. This option is ideal for a software as a service (SaaS) application or provider because you can manage and monitor performance in a simplified way for many databases.

SQL Managed Instance pools allow you to host multiple managed instances and share resources. You can **pre-provision compute resources**. Doing so can reduce overall deployment time to make migrations easier.

Azure SQL Deployments

SQL virtual machines

Best for migrations and applications requiring OS-level access



- SQL Server and OS server access
- Expansive SQL and OS version support
- Automated manageability features for SQL Server

Managed instances

Best for most lift-and-shift migrations to the cloud



- SQL Server surface area (vast majority)
- Native virtual network support
- Fully managed service



- Pre-provision compute resources for migration
- Enables cost-efficient migration
- Ability to host smaller instances (2Vcore)
- Fully managed service
- In public preview

Databases

Best for modern cloud applications. Hyperscale and serverless options are available



- Hyperscale storage (up to 100TB)
- Serverless compute
- Fully managed service



- Resource sharing between multiple databases to price optimize
- Simplified performance management for multiple databases
- Fully managed service

Further Reading

<https://docs.microsoft.com/en-us/azure/cosmos-db/>



Any Questions?